# Weekwise Study Notes on Cloud Services and Platforms

Based primarily on trimester notes from the cloud resource PDF

`ml-workbook/bits-pilani/trimester-2/notes/cloud-services-and-platforms`

Secondary strengthening from `my resources/cloud resource`

Especially the practice paper in `Cloud Services & Platforms.docx`

| | |
|---|---|
| Coverage: | Week 1 to Week 4 in repository order |
| Primary Source: | `my resources/cloud resource/Full_Trimester_Notes.pdf` |
| Secondary Source: | `Cloud Services & Platforms.docx` for exam pattern |
| Goal: | Detailed, beginner-friendly, and exam-ready notes |
| Output File: | `cloud_weekwise_study_notes.tex` |

Prepared on April 18, 2026

# Contents

# 1 How to Use These Notes

These notes preserve the original week order of the repository and turn the course material into a connected study document. The repo content is the main source of truth. The resource folder is used only to strengthen exam preparation and fill gaps that a student may face in a written paper.

The design of these notes is intentionally exam-oriented. For most major concepts, the notes try to answer the following:

- What is the concept?
- Why does it matter in cloud computing?
- How does it work technically?
- What trade-off does it introduce?
- Where is it used in practice?
- How is it likely to be asked in an exam?

Suggested study method:

1. Read one week at a time for concept clarity.
2. Before exams, focus on tables, exam focus notes, and section-end recaps.
3. Use the final question bank and last-minute revision section for quick revision.

# 2 Exam Pattern from the Practice Paper

The practice paper in `Cloud Services & Platforms.docx` from `my resources/cloud resource` is short, but it reveals the style of questioning very clearly. It does not behave like a one-line memory test. It expects structured explanation, comparison, architectural reasoning, and practical cloud judgment.

## 2.1 Observed Question Psychology

The paper suggests the examiner is likely to prefer:

- comparison questions such as OpEx vs CapEx or horizontal vs vertical scaling,
- architecture reasoning questions such as how to create a secure controlled environment,
- migration strategy questions such as rehosting, replatforming, and refactoring,
- operational workflow questions such as container lifecycle or CI/CD stages,
- cloud security configuration questions that mix theory with design logic,
- answers that explain not only “what” but also “when to use” and “why it helps.”

## 2.2 Implication for Preparation

A safe exam preparation strategy is:

- learn definitions,
- learn one simple example for each major concept,
- learn at least one comparison table for each important pair,
- learn one architecture-style answer format for security, scaling, and governance questions,
- learn a few supplementary DevOps-connected topics because the practice paper blends cloud and operations thinking.

# 3 Week 1: Foundations of Cloud Computing

## 3.1 Week 1 Roadmap

Week 1 establishes the basic language of cloud computing. It moves from the historical evolution of computing to cloud definition, key characteristics, service models, deployment models, real-world applications, and adoption trends. If Week 1 is weak, later topics become harder because many exam answers depend on clear basic distinctions.

## 3.2 Introduction to the Subject

Cloud computing is best understood as the delivery of computing capabilities over the internet in a flexible and pay-per-use manner. Instead of first buying servers, installing them, cooling them, securing them, and maintaining them for years, an organization can consume infrastructure and services when needed.

This shift matters because computing stopped being only a physical asset problem and became a service consumption problem. That is why cloud is often compared with utilities like electricity. You do not build a power plant for every small office. You consume power as needed. Cloud computing applies a similar idea to computing resources.

## 3.3 Evolution of Computing Toward the Cloud

The repository emphasizes that cloud did not appear suddenly. It evolved through several stages:

1. Mainframe era: centralized computation with terminals.
2. Personal computing era: computation moved to individual machines.
3. Client-server era: applications split across client and server systems.
4. Virtualization era: one physical machine could host many isolated virtual machines.

5. Cloud era: infrastructure and platforms became on-demand services.

**Why this evolution matters:**

- It explains why virtualization is a technical foundation of cloud.
- It shows that cloud is not only "remote servers" but a mature service-delivery model.
- It clarifies why concepts like pooling, multi-tenancy, and elasticity are possible.

**Intuitive interpretation:** Earlier computing models required heavy planning before usage. Cloud reverses that sequence. Usage comes first, capacity provisioning happens dynamically behind the scenes.

## 3.4 Definition of Cloud Computing

A working exam-safe definition is:

*Cloud computing is the on-demand delivery of computing services such as compute power, storage, networking, databases, and software over the internet with pay-as-you-use pricing and rapid elasticity.*

**Important terms in the definition:**

- **On-demand**: resources can be provisioned whenever needed.
- **Services**: not only hardware, but usable capabilities.
- **Over the internet**: access is network-based.
- **Pay-as-you-use**: cost is tied to consumption.
- **Elasticity**: capacity can move with workload demand.

**Why the definition matters in exams:** Definitions alone rarely get full marks. A better answer explains how cloud differs from traditional infrastructure:

- no large upfront procurement,
- no need to manually provision everything in advance,
- faster deployment,
- better alignment between cost and usage.

## 3.5 Key Characteristics of Cloud Computing

The week-wise notes emphasize three central traits strongly: scalability, elasticity, and on-demand self-service. For completeness, it is useful to state the broader characteristic set usually associated with cloud:

- on-demand self-service,
- broad network access,
- resource pooling,
- rapid elasticity,
- measured service.

### 3.5.1 On-Demand Self-Service

Users can provision resources without waiting for long manual procurement or infrastructure teams. This matters because speed of provisioning directly affects speed of experimentation and delivery.

**Example:** A development team can create a test server in minutes instead of raising a ticket and waiting days.

### 3.5.2 Broad Network Access

Cloud services are accessed over standard networks and interfaces. This supports remote work, distributed teams, and multi-location usage.

### 3.5.3 Resource Pooling

The provider pools infrastructure and serves many customers efficiently. Customers do not usually know the exact machine on which their workload runs. They consume capacity from a shared infrastructure pool.

### 3.5.4 Rapid Elasticity

Resources can expand or shrink rapidly. This is one of the strongest practical reasons to use cloud for variable workloads.

### 3.5.5 Measured Service

Usage is monitored and billed. This creates both an advantage and a discipline:

- advantage because you avoid idle capital investment,
- discipline because bad cost control can cause unnecessary bills.

## 3.6 Why Cloud Matters Today

The repository lists six strong adoption drivers. These are valuable for long answers.

### 3.6.1 1. Cost Savings

Cloud reduces large upfront capital expenditure on servers, racks, power, cooling, and maintenance. It shifts spending toward operational expenditure.

**Why this matters:**

- startups can begin without data center investment,
- enterprises can avoid over-buying hardware for uncertain future demand,
- variable demand becomes financially easier to manage.

### 3.6.2 2. Faster Innovation and Time-to-Market

Teams can provision infrastructure, databases, queues, and analytics platforms quickly. This reduces the time between idea and deployment.

### 3.6.3 3. Scalability and Elasticity

Cloud allows applications to handle demand spikes more gracefully. This is especially important for campaigns, streaming launches, exam portals, ticketing systems, and festival season sales.

### 3.6.4 4. Global Reach

Cloud providers offer many regions. Applications can be deployed closer to users, reducing latency and improving experience.

### 3.6.5 5. Reliability and High Availability

Cloud providers expose architectural patterns such as multi-zone deployment, failover, load balancing, and backups. These make resilient design easier than in many traditional environments.

### 3.6.6 6. Security and Compliance Enablement

Providers invest heavily in physical and platform-level security. However, this does not remove customer responsibility. It changes the way responsibility is divided.

**Exam Focus**

In a 5-mark answer on "Why cloud matters today," do not just list points. Define cloud in one line, explain any four or five drivers, and attach a business example such as remote collaboration, fintech launch, or seasonal e-commerce traffic.

## 3.7 Cloud Service Models: IaaS, PaaS, SaaS

This is one of the most predictable exam topics. The real concept is responsibility distribution.

### 3.7.1 IaaS: Infrastructure as a Service

In IaaS, the provider manages foundational infrastructure such as networking, servers, storage, and virtualization. The customer still manages the operating system, runtime, application, and data.

**Why it exists:** Some teams need control over system configuration, operating system tuning, and custom software stacks.

**Example:** Running a custom Linux-based enterprise application on virtual machines.

**Advantage:** High control and flexibility.

**Disadvantage:** Higher operational burden because patching, hardening, and maintenance remain with the customer.

### 3.7.2 PaaS: Platform as a Service

In PaaS, the provider manages the infrastructure and much of the platform layer. The customer focuses mainly on application code and data.

**Why it matters:** It reduces infrastructure distraction and improves developer productivity.

**Example:** Deploying a web application to a managed application platform without manually patching the OS.

**Advantage:** Faster development and easier deployment.

**Disadvantage:** Less low-level control and sometimes platform lock-in.

### 3.7.3 SaaS: Software as a Service

In SaaS, the provider manages almost the complete stack. The customer mainly uses the software and performs business-level configuration.

**Example:** Email, CRM, HR software, collaboration suites.

**Advantage:** Fast adoption and low operational effort.

**Disadvantage:** Least customization and lower control over internal implementation.

### 3.7.4 Control vs Convenience Trade-off

The deeper concept behind these models is:

$$\text{More control} \Rightarrow \text{more responsibility}, \qquad \text{more convenience} \Rightarrow \text{less control}$$

| Model | Control Level | Customer Effort | Best Fit |
|---|---|---|---|
| IaaS | High | High | Legacy migration, custom stacks, special security or OS needs |
| PaaS | Medium | Medium | Fast app development and deployment |
| SaaS | Low | Low | Commodity business functions and rapid adoption |

**Common Confusion Points**

Students often memorize the three names without understanding the real axis of comparison. The core question is always: who manages what?

## 3.8 Shared Responsibility Model

The repository stresses that this topic is critical because many real cloud incidents are not due to provider failure but customer misconfiguration.

### 3.8.1 Core Principle

- The provider is responsible for **security of the cloud**.
- The customer is responsible for **security in the cloud**.

### 3.8.2 Provider Responsibilities

- physical data center security,
- hardware protection,
- power and cooling,
- core networking and virtualization foundations,
- platform availability foundations.

### 3.8.3 Customer Responsibilities

- identity and access management,
- data classification and protection,
- correct permission settings,
- network rules and firewall intent,

- encryption choices and key management,
- OS patching in IaaS,
- secure application design.

**Simple example:** If a storage bucket becomes public due to a bad access setting, that is usually not the provider's mistake. It is a customer-side configuration failure.

**Why the model matters:**

- it prevents false assumptions such as "cloud provider handles all security,"
- it explains why service model choice changes security burden,
- it is directly relevant to compliance, IAM, encryption, and architecture answers.

## 3.9 Public, Private, Hybrid, and Multi-Cloud

Deployment model questions test whether a student can connect business constraints with architecture choice.

### 3.9.1 Public Cloud

Resources are hosted by a third-party cloud provider and delivered over the internet.

**Strengths:**

- high scalability,
- fast provisioning,
- broad service range,
- reduced infrastructure ownership.

**Weaknesses:**

- lower physical control,
- possible compliance concerns,
- dependence on provider ecosystem.

### 3.9.2 Private Cloud

Cloud-like infrastructure is dedicated to one organization.

**Strengths:**

- more control,

- easier alignment with certain compliance or governance needs,
- stronger customization options.

**Weaknesses:**

- higher management burden,
- reduced elasticity compared with large public cloud providers,
- higher cost per unit of flexibility.

### 3.9.3 Hybrid Cloud

Hybrid cloud combines private and public environments in one architecture. Sensitive workloads may stay private, while bursty or customer-facing workloads use the public cloud.

**Example:** A hospital keeps regulated records in private infrastructure while running appointment booking in public cloud.

### 3.9.4 Multi-Cloud

Multi-cloud means using multiple public cloud providers for different workloads.

**Why organizations do this:**

- avoid vendor lock-in,
- choose best-in-class services,
- distribute provider risk,
- improve negotiation leverage.

| Aspect | Hybrid Cloud | Multi-Cloud |
|---|---|---|
| Composition | Private + public cloud integration | Multiple public cloud providers |
| Main driver | Balance control and scalability | Diversification and best-of-breed service choice |
| Typical users | Regulated industries and large enterprises | Global tech firms and provider-diversified organizations |

## 3.10 Choosing the Right Cloud Model

This topic is usually not about memorizing one "best" model. It is about context.

**Selection logic:**

- choose public cloud for agility and scale,
- choose private cloud when control and regulation dominate,
- choose hybrid when both control and elasticity are needed,
- choose multi-cloud when provider diversification or specialized service selection matters.

**Exam answer structure:**

1. state the business requirement,
2. identify the main risk or constraint,
3. justify the model choice,
4. mention one trade-off.

## 3.11 Real-World Applications of Cloud

Cloud is not an abstract infrastructure idea. It underpins many daily services:

- video streaming,
- e-commerce,
- online collaboration,
- ride sharing,
- file synchronization,
- analytics and AI services,
- backups and disaster recovery.

The value of cloud differs by workload:

- streaming needs scale and global distribution,
- ride sharing needs real-time low-latency coordination,
- file sync needs durability and availability,
- enterprise collaboration needs remote access and reliability.

## 3.12 Case Studies: Netflix, Uber, Dropbox

### 3.12.1 Netflix

Netflix demonstrates cloud value in global content delivery, unpredictable spikes, and high availability expectations. The deeper lesson is that cloud supports large-scale demand volatility and geographically distributed user bases.

### 3.12.2 Uber

Uber shows the importance of low-latency processing, geographic expansion support, and continuous real-time matching. This is a good example for cloud answers that require operational speed rather than only storage or compute scale.

### 3.12.3 Dropbox

Dropbox highlights storage durability, synchronization, collaboration, and secure access. This case is useful in answers about object storage, availability, and cloud-enabled productivity.

| Company | Main Cloud Need | Core Cloud Benefit |
|---|---|---|
| Netflix | Global streaming and traffic spikes | Elastic scale and content distribution |
| Uber | Real-time matching and notifications | Low-latency distributed processing |
| Dropbox | Durable sync and collaboration | Reliable storage and always-available access |

## 3.13 Current Industry Trends in Cloud Adoption

The repository closes Week 1 with industry trends. A good synthesis answer includes:

- stronger cloud adoption across industries,
- hybrid and multi-cloud growth,
- more managed services,
- increasing security and compliance focus,
- sustainability concerns entering architecture conversations,
- cloud as a business enabler rather than just hosting.

### Key Concept Recap

Week 1 builds the conceptual base of the whole course. The most repeated themes are service delivery, elasticity, responsibility boundaries, and business-driven model selection.

### Last-Minute Revision Bullets

- Cloud = on-demand services over the internet with pay-per-use and elasticity.
- IaaS, PaaS, SaaS differ mainly by responsibility split.

- Shared responsibility means provider secures the cloud; customer secures usage in the cloud.
- Hybrid includes private + public integration; multi-cloud means multiple public providers.
- Netflix, Uber, Dropbox are useful real-world examples for long answers.

# 4 Week 2: Cloud Architecture Principles and Quality Attributes

## 4.1 Week 2 Roadmap

Week 2 focuses on architecture quality. The central themes are scalability, elasticity, availability, fault tolerance, multi-tenancy, reliability, performance, trade-offs, and the architectural frameworks used across major cloud providers.

## 4.2 Scalability

Scalability is a system's ability to handle increasing load while maintaining acceptable performance. This is about growth capacity.

### 4.2.1 Vertical Scaling

Vertical scaling means making one machine stronger. You increase CPU, RAM, or storage performance.

**Advantages:**

- simple to understand,
- often minimal application redesign,
- common for traditional databases and monolithic systems.

**Limitations:**

- hardware ceiling exists,
- may require downtime,
- high-end machines are costly,
- still leaves a strong dependency on a single node.

### 4.2.2 Horizontal Scaling

Horizontal scaling means adding more machines and distributing work across them.

**Advantages:**

- better cloud fit,
- better fault tolerance,
- no fixed theoretical upper limit in the same way as vertical scaling,
- easier to support large-scale internet-facing systems.

**Requirement:** The system architecture must support distribution. That is why stateless design, load balancing, and external session storage often appear with horizontal scaling.

## 4.3 Elasticity

Elasticity extends scalability by adding automatic up-scaling and down-scaling. This is not just a technical convenience. It is a financial control mechanism.

**Core idea:**

$$\text{Elasticity} = \text{Scalability} + \text{automatic response to demand variation}$$

**Why elasticity matters:**

- avoids over-provisioning,
- reduces under-provisioning risk,
- aligns resource usage with actual demand,
- supports unpredictable workloads.

**Step-by-step control loop:**

1. Monitor a metric such as CPU load, requests per second, or queue depth.
2. Compare it with a threshold or policy.
3. Add instances when demand rises.
4. Remove instances when demand falls.
5. Repeat continuously.

| Aspect | Scalability | Elasticity |
|---|---|---|
| Main question | Can the system grow? | Can it grow and shrink automatically? |
| Automation | Optional | Required |
| Direction | Usually upward growth emphasis | Upward and downward adjustment |
| Cost impact | May leave idle capacity | Stronger cost control |

**Exam Focus**

If the exam asks horizontal vs vertical scaling, include architecture consequences. If the exam asks scalability vs elasticity, explicitly mention automation and cost behavior.

## 4.4 High Availability vs Fault Tolerance

Both topics concern failure handling, but they differ in strictness.

### 4.4.1 High Availability

High availability means the system remains available most of the time and recovers quickly from failures. Brief interruptions may still happen.

**Typical building blocks:**

- redundancy,
- load balancing,
- health checks,
- automatic failover,
- multi-zone deployment.

### 4.4.2 Fault Tolerance

Fault tolerance aims to continue operating with no visible interruption even during component failure. This usually requires active-active designs, synchronized data, and elimination of single points of failure.

**Why fault tolerance is rare:**

- expensive,
- operationally complex,
- justified only for truly mission-critical systems.

| Aspect | High Availability | Fault Tolerance |
|---|---|---|
| User-visible downtime | Small interruption may occur | No visible interruption intended |
| Strategy | Recover quickly | Absorb failure instantly |
| Cost | Moderate | High |
| Typical use | Most business systems | Banking, trading, medical, critical control systems |

## 4.5 Multi-Tenancy

Multi-tenancy means multiple customers or tenants share the same underlying cloud infrastructure while remaining logically isolated.

**Why this matters:**

- it improves provider efficiency,
- it lowers cost,
- it makes large-scale cloud service possible.

**Deep technical idea:** Sharing does not mean everyone sees everyone else's data. Isolation is enforced by virtualization, identity systems, access control, network separation, and service-level design.

**Simple example:** Many customers may store data in the same cloud storage service, but access control ensures each customer can access only their own objects.

**Risk to remember:** Isolation failures are severe because multi-tenancy only works if boundaries remain strong.

## 4.6 Reliability in Cloud Architecture

Reliability means the system performs its intended function correctly and consistently over time. In cloud terms, reliability is not only about uptime. It includes recovery capability, backup strategy, resilience against failure, and operational predictability.

**Typical reliability enablers:**

- redundancy,
- replication,
- backups,
- failover,
- health monitoring,
- tested recovery procedures.

**Exam-ready insight:** Availability and reliability are related but not identical. A system may be frequently available yet unreliable if it produces errors, loses data, or fails unpredictably.

## 4.7 Performance Considerations

Performance in cloud architecture is about achieving acceptable speed, throughput, and responsiveness for the workload.

**Factors influencing performance:**

- compute sizing,
- storage type,
- network design,
- caching strategy,
- latency to users,
- database selection,
- architectural patterns such as asynchronous processing.

**Important exam idea:** Performance efficiency does not mean maximum power everywhere. It means appropriate resource selection for the workload.

## 4.8 Trade-Offs in Cloud Architecture

Week 2 repeatedly implies that cloud design is about balancing competing goals. Typical trade-offs include:

- cost vs performance,
- availability vs complexity,
- flexibility vs operational burden,
- control vs speed,
- security strictness vs ease of access,
- regional reach vs operational cost.

**Why this matters in exams:** Many descriptive questions become stronger if you explicitly mention trade-offs. It shows architectural maturity.

## 4.9 The Six Pillars of Well-Architected Thinking

The notes introduce the six-pillar perspective across frameworks. These pillars are useful because they organize architecture answers.

| Pillar | Main Concern |
|---|---|
| Operational Excellence | Running, monitoring, and improving systems effectively |
| Security | Protecting systems, data, and identities |
| Reliability | Recovery, resilience, and consistency of operation |
| Performance Efficiency | Right-sizing and efficient use of resources |
| Cost Optimization | Avoiding waste and aligning spending to value |
| Sustainability | Reducing environmental impact and idle resource waste |

### 4.9.1 Operational Excellence

Focuses on automation, observability, incident response, and continuous improvement. This pillar pushes teams away from manual, fragile operations.

### 4.9.2 Security

Focuses on least privilege, encryption, continuous monitoring, secure configuration, and defense against misconfiguration.

### 4.9.3 Reliability

Focuses on fault recovery, failover, testing, backups, and resilient architecture.

### 4.9.4 Performance Efficiency

Focuses on correct resource selection, stateless design, caching, asynchronous patterns, and managed services when helpful.

### 4.9.5 Cost Optimization

Focuses on using only what is needed, removing unused resources, and continuously tracking cost.

### 4.9.6 Sustainability

Focuses on using compute and storage responsibly, reducing idle usage, and designing systems that waste less energy.

## 4.10 AWS, Azure, and Google Cloud Architectural Guidance

The repo mentions AWS Well-Architected, Azure Architecture Center, and the Google Cloud Architecture Framework. The exam is unlikely to require memorizing vendor-specific wording in detail. What matters more is recognizing the common design philosophy:

- secure by design,
- automate operations,
- build for failure,
- observe systems continuously,
- optimize cost and performance,
- think globally when needed.

## 4.11 Cross-Cloud Architecture Principles

This topic is especially important because it shifts thinking from vendor names to universal architecture principles.

**Core universal principles from the repo:**

1. Design for failure.
2. Automate everything practical.
3. Prefer managed services when they reduce operational burden.
4. Build stateless application layers where possible.
5. Apply least privilege and continuous monitoring.
6. Treat cost optimization as ongoing work.
7. Maintain observability through logs, metrics, and traces.
8. Design for global scale when the workload requires it.

**Why this matters:** Tools differ across AWS, Azure, and GCP, but these principles remain stable. That is why cloud architecture knowledge should be principle-first, service-name second.

### Key Concept Recap

Week 2 teaches that cloud systems are judged by quality attributes, not only by whether they run. A good answer should show the difference between growth, resilience, efficiency, and governance.

**Last-Minute Revision Bullets**

- Scalability is growth capacity; elasticity adds automatic up/down response.
- Horizontal scaling fits cloud-native systems better than vertical scaling.
- High availability allows quick recovery; fault tolerance aims for no visible downtime.
- Multi-tenancy means shared infrastructure with logical isolation.
- The six pillars provide a strong structure for architecture answers.
- Cross-cloud principles matter more than vendor branding.

# 5 Week 3: Comparing Major Cloud Providers and Service Categories

## 5.1 Week 3 Roadmap

Week 3 moves from principles to provider comparison. The main theme is that AWS, Azure, and Google Cloud often provide equivalent categories of services under different names. The week also introduces provider strengths, cost and compliance differences, enterprise fit, career paths, and workload alignment.

## 5.2 Global Footprint

Global footprint means the worldwide spread of regions, zones, networks, and service presence offered by a cloud provider.

**Why global footprint matters:**

- reduces latency by deploying closer to users,
- supports data residency and compliance choices,
- improves resilience through geographic distribution,
- affects expansion speed into new markets.

**Strategic caution:** More regions do not automatically mean a better answer for every workload. Additional regions also increase replication, governance, and cost complexity.

## 5.3 Service Equivalents Across Providers

One of the most important Week 3 lessons is that different names often map to similar core cloud concepts.

| Category | AWS | Azure | Google Cloud |
|---|---|---|---|
| Compute VM | EC2 | Virtual Machine | Compute Engine |
| Object Storage | S3 | Blob Storage | Cloud Storage |
| Managed Relational DB | RDS | SQL Database | Cloud SQL |
| Private Network | VPC | Virtual Network | VPC |

### 5.3.1 Why this mapping matters

- it reduces fear of switching providers,
- it helps in architecture interviews and exams,
- it makes cloud learning category-based rather than brand-based.

## 5.4 Compute Services

Compute services provide the machines or execution environments in which applications run.

**Deep explanation:** Compute is where application logic actually executes. It gives CPU, memory, storage attachment, network interfaces, and control over the runtime environment. With VM-based compute, the customer often controls the operating system and software stack.

**Why compute choice matters:**

- affects flexibility,
- affects operational burden,
- affects pricing model,
- affects scalability behavior.

## 5.5 Storage Services

Object storage is emphasized strongly in the repo. It stores unstructured data such as images, documents, logs, backups, and media files.

**Important point:** Object storage is not the same as a traditional file system mounted like a disk. It is typically accessed via APIs and is designed for massive durability and scale.

**Good exam example:** Store user-uploaded images, video files, archived logs, and backup snapshots in object storage.

## 5.6 Managed Database Services

Managed relational database services reduce operational burden by handling tasks such as:

- patching,
- backups,
- replication,
- failover,
- monitoring.

**Why managed databases matter:** Databases are stateful and operationally expensive to manage well. Managed services let teams focus more on schema design, queries, and application behavior.

## 5.7 Networking Services

Private cloud networking is the foundation of secure architecture. Equivalent services such as AWS VPC, Azure Virtual Network, and GCP VPC all create logically isolated networking environments.

**Typical networking building blocks:**

- IP ranges,
- subnets,
- routing,
- gateways,
- firewall rules,
- controlled access paths.

**Exam-safe message:** Even a strong application can be made insecure by weak network design.

## 5.8 Provider Strengths and Positioning

The repo suggests a broad high-level view:

- **AWS**: broadest service range, strong flexibility, deep ecosystem.
- **Azure**: strong enterprise and Microsoft ecosystem alignment.
- **Google Cloud**: strong data, analytics, and network performance reputation.

These are not absolute truths for every scenario, but they are useful summary points.

## 5.9 Cost, Compliance, and Global Availability

This is one of the best Week 3 theory sections because it connects architecture with business decisions.

### 5.9.1 Cost

Cloud uses a pay-as-you-go model. This removes the need for large upfront hardware investment, but it introduces a new discipline: active cost control.

**Key insight:** Idle resources are not harmless. They become recurring cost leakage.

### 5.9.2 Compliance

Compliance concerns how data is stored, processed, protected, and governed. Moving to cloud does not automatically make an application compliant. The provider may offer compliant infrastructure, but the customer's usage and configuration determine final compliance posture.

### 5.9.3 Global Availability

Global availability concerns how well the provider can support low-latency delivery and resilient deployment across geographies. This is tied to region count, network design, and operational readiness.

| Factor | AWS | Azure | Google Cloud |
|---|---|---|---|
| Cost view | Broad flexibility, needs governance | Strong fit for Microsoft-heavy enterprises | Known for helpful discount patterns on steady workloads |
| Compliance view | Wide coverage | Strong enterprise and public-sector fit | Strong data and analytics privacy orientation |
| Global view | Large footprint | Large enterprise-oriented footprint | Fewer regions than AWS but strong global network |

## 5.10 Enterprise Integration Essentials

Week 3 also points toward enterprise readiness. Large organizations rarely choose a cloud provider based only on raw infrastructure. They care about:

- identity integration,
- governance,

- security controls,
- compatibility with existing tools,
- migration path from current systems,
- financial and compliance alignment.

## 5.11 Scalability and Workload Fit

Not every provider has the same strategic fit for every workload. A better way to think is:

- identify workload nature,
- identify business constraints,
- identify ecosystem dependencies,
- then choose provider and architecture.

**Examples:**

- Microsoft-centric enterprise workloads often align naturally with Azure.
- Data-heavy analytics teams may find Google Cloud attractive.
- Teams needing the widest menu of services may prefer AWS.

## 5.12 Certification Paths and Career Roles

These topics may not dominate the theory paper, but they help in viva or short notes.

**Certification logic:** Certification paths usually move from foundational understanding to role-specific specializations such as architecture, administration, security, or data.

**Cloud career roles commonly include:**

- cloud architect,
- cloud administrator,
- DevOps engineer,
- security engineer,
- site reliability engineer,
- data engineer,
- platform engineer.

#### Key Concept Recap

Week 3 teaches translation across providers. The safest way to remember it is: compute, storage, database, and networking are universal categories even when branding differs.

#### Last-Minute Revision Bullets

- Learn service equivalence by category, not just by vendor name.
- Object storage is API-based storage for unstructured data.
- Managed databases reduce ops burden by handling backup, patching, and failover.
- Global footprint matters for latency, resilience, and compliance.
- Provider choice is a business-architecture decision, not a popularity contest.

# 6 Week 4: AWS Foundations, IAM, and Governance

## 6.1 Week 4 Roadmap

Week 4 is more operational and security-oriented. It introduces AWS global infrastructure, account structure, billing basics, IAM users and groups, roles, policies, MFA, access keys, and AWS Organizations. This week is highly exam-relevant because IAM and governance questions are common and easy to ask in both theory and applied form.

## 6.2 AWS Global Infrastructure

AWS global infrastructure is usually described through regions, availability zones, and edge-related presence.

### 6.2.1 Region

A region is a separate geographic area containing multiple data center zones. Choosing a region affects latency, compliance, and disaster planning.

### 6.2.2 Availability Zone

An availability zone is an isolated location within a region. Deploying across multiple zones improves availability and reduces the risk that one data center issue will take down the entire application.

### 6.2.3 Why this matters

This infrastructure model explains why multi-AZ design appears so frequently in high availability answers.

## 6.3 Account Structure and Billing Basics

An AWS account is both a security boundary and a billing boundary. This is an important practical idea.

**Why account structure matters:**

- it isolates resources,
- it separates environments,
- it improves governance,
- it supports cost tracking,
- it reduces blast radius.

**Billing basics to remember:**

- cloud cost is usage-based,
- bad cleanup creates real cost,
- governance and tagging help attribution,
- multi-account strategy improves organizational control.

## 6.4 IAM: Identity and Access Management

IAM decides who can access AWS and what they can do. This is the control plane of AWS security.

### 6.4.1 IAM Users

An IAM user represents a single identity, usually one person. One user should map to one real identity for accountability.

**Why shared users are bad:**

- loss of individual accountability,
- harder auditing,
- security confusion,
- poor operational practice.

### 6.4.2 IAM Groups

Groups are collections of users. They simplify permission management by allowing policies to be attached once at the group level.

**Example:** All developers can be placed in one group with shared developer permissions.

### 6.4.3 Least Privilege

Least privilege means granting only the permissions required to perform a job and nothing more.

**Why it matters:**

- reduces damage from compromised credentials,
- reduces accidental misuse,
- improves security posture,
- supports compliance and auditability.

| Aspect | IAM User | IAM Group |
|---|---|---|
| Represents | One identity | Collection of users |
| Has credentials | Yes | No |
| Best use | Individual accountability | Permission management at scale |

## 6.5 IAM Roles and Trust

Roles are different from users. A role is an identity with permissions that can be assumed temporarily by another trusted entity.

**Why roles matter:**

- reduce need for long-term credentials,
- support secure machine-to-machine access,
- support cross-service and cross-account delegation,
- improve security over static access keys in many cases.

**Trust relationship:** A role must define who is allowed to assume it. This is the trust side of the model. Permissions define what the role can do after assumption.

**Exam distinction:** Users are long-lived identities. Roles are assumable permission bundles for temporary or delegated access.

## 6.6 IAM Policies and Evaluation Logic

Policies are JSON-based rule documents that define allowed or denied actions on resources.

### 6.6.1 Core components of a statement

- Effect,
- Action,
- Resource,
- optional Condition.

### 6.6.2 Evaluation logic

AWS evaluates requests using a security-first model:

1. Check for explicit deny.
2. If explicit deny exists, reject immediately.
3. If no deny, check for explicit allow.
4. If allow exists, permit the action.
5. Otherwise deny by default.

$$\text{Explicit Deny} > \text{Explicit Allow} > \text{No Match}$$

$$\text{No Match} \Rightarrow \text{Implicit Deny}$$

**Why this topic is exam-friendly:** It allows short theory questions, scenario questions, and direct logic questions.

**Example:** If one policy allows reading from a storage bucket and another policy explicitly denies deletion, the user may read but cannot delete.

## 6.7 Creating Users and Groups

Operationally, the correct flow is:

1. create a user for each person,
2. create groups by function,
3. attach permissions to groups,
4. add users to groups,
5. review permissions regularly.

This is not just a setup step. It is a maintainability pattern.

## 6.8 MFA and Password Policies

Multi-factor authentication adds another verification factor beyond the password. Password policies define strength, rotation expectations, and account hygiene rules.

**Why MFA matters:**

- passwords can be guessed, reused, leaked, or phished,
- MFA dramatically reduces risk from password-only compromise,
- admin or privileged identities should never rely on passwords alone.

**Password policy goals:**

- stronger passwords,
- fewer weak default habits,
- lower brute-force and credential stuffing risk.

## 6.9 IAM Auditing and Credential Hygiene

Week 4 also points to auditing and password review practices. This is important because security does not end at account creation. Good governance includes:

- reviewing who has access,
- checking unused credentials,
- enforcing MFA,
- rotating risky secrets,
- removing stale accounts.

## 6.10 Access Keys and Security Best Practices

Access keys are used for programmatic access to AWS, especially from external tools and the CLI.

**Structure:**

- access key ID,
- secret access key.

**Important operational rule:** The secret key is shown only once at creation.

**Typical uses:**

- AWS CLI,

- automation tools,
- third-party integrations such as infrastructure automation pipelines.

**Security best practices:**

- never hard-code keys in source code,
- never share keys,
- deactivate and delete keys when compromised,
- prefer roles over static keys when architecture allows,
- remove unused keys.

| Method | Main Use | Security View |
|---|---|---|
| Console login | Human web access | Safer when combined with MFA |
| Access keys | Programmatic access | Sensitive; must be rotated and protected |
| Roles | Temporary delegated access | Preferred where possible |

## 6.11 AWS Organizations

AWS Organizations supports management of multiple AWS accounts under one centrally governed structure.

### 6.11.1 Main building blocks

- management account,
- member accounts,
- organizational units,
- service control policies,
- consolidated billing.

### 6.11.2 Management Account

The management account controls billing, governance, and organization-level policy. It should not be used casually for application workloads.

### 6.11.3 Member Accounts

Member accounts host actual workloads such as development, testing, and production systems.

### 6.11.4 Organizational Units

OUs are logical groupings of accounts, often by function, team, or environment.

### 6.11.5 Service Control Policies

SCPs are guardrails. They do not grant permissions. They limit the maximum permissions available within accounts or OUs.

**Key exam line:** *SCPs restrict; IAM policies grant.*

### 6.11.6 Consolidated Billing

One bill can cover all member accounts, improving financial visibility and governance.

### Exam Focus

For AWS Organizations answers, always distinguish management account, member account, and SCP clearly. The common mistake is saying SCPs grant permissions. They do not.

### Key Concept Recap

Week 4 teaches that cloud security and governance are not optional add-ons. Identity design, permission evaluation, credential hygiene, and multi-account control are core parts of cloud architecture.

### Last-Minute Revision Bullets

- IAM decides who can do what.
- One IAM user should map to one real identity.
- Groups simplify permission management; least privilege reduces risk.
- Roles are assumable identities for temporary or delegated access.
- Explicit deny always wins in policy evaluation.
- MFA is essential for secure access.
- Access keys are sensitive and should be rotated or avoided when roles are better.
- AWS Organizations enables multi-account governance with SCP guardrails.

# 7 Week 5: Compute Services, Virtual Machines, and Elastic Compute Patterns

## 7.1 Week 5 Focus

The PDF shifts in Week 5 from foundational cloud thinking to compute design. The main emphasis is AWS EC2, its components, block storage attachment, scaling, and traffic distribution. This week matters because most cloud architecture questions eventually depend on compute behavior.

## 7.2 What EC2 Really Is

Amazon EC2 stands for Elastic Compute Cloud. It is AWS's virtual machine service. An EC2 instance is a virtual server running in AWS infrastructure.

**Definition:** EC2 is a compute service that lets users create and run virtual machines with chosen CPU, memory, storage, networking, and operating system settings.

**How it works:**

1. You choose a machine image.
2. You choose an instance type.
3. You attach networking and security settings.
4. AWS provisions a virtual machine on its infrastructure.
5. You connect, configure software, and run workloads.

**Why it is used:**

- It gives strong control over the operating system.
- It supports custom software stacks.
- It is suitable for lift-and-shift migration.
- It supports predictable virtual machine style operations.

**Where it is used:**

- application servers,
- web servers,
- development and testing machines,
- self-managed databases,
- batch processing,

- legacy workloads that need server-level control.

**Simple analogy:** If Lambda is event-driven taxi usage, EC2 is renting a vehicle that stays available and under your control until you stop it.

## 7.3 Core EC2 Components

The PDF repeatedly explains EC2 through three building blocks: AMI, instance type, and key pair.

### 7.3.1 AMI: Amazon Machine Image

An AMI is the template from which an instance is launched. It contains the operating system and may also include application software or configuration.

**Why it matters:** The AMI decides the initial software environment. Two instances with the same instance type may behave differently if their AMIs differ.

**Use case:** A team can maintain one hardened Linux AMI for secure application deployment across multiple environments.

### 7.3.2 Instance Type

The instance type defines the hardware profile. It determines CPU, RAM, network performance, and sometimes specialized hardware such as GPUs.

**Why it matters:** Choosing the wrong type causes either waste or poor performance. The PDF frames this as a workload-fit decision, not a memorization exercise.

### 7.3.3 Key Pair

A key pair is used for secure login to Linux instances through SSH. AWS typically does not use password-based access for EC2 by default.

**Why it matters:** It is a security mechanism and an operational access mechanism. Losing the key pair can complicate access recovery.

| Component | Role in Launching an Instance |
|---|---|
| AMI | Defines starting software image and operating system |
| Instance Type | Defines compute capacity and hardware profile |
| Key Pair | Enables secure administrative access |

## 7.4 Security Groups and Traffic Control

The PDF places security groups early in Week 5 because compute security begins at the network edge of the instance.

**Definition:** A security group is a virtual firewall attached to an instance or resource. It controls allowed inbound and outbound traffic.

**Important property:** Security groups are stateful. If inbound traffic is allowed and a connection is established, the return traffic is automatically allowed.

**Why it is used:**

- to restrict open ports,
- to reduce attack surface,
- to separate web, application, and database tiers,
- to implement least-access networking.

**Example:**

- Web server security group allows HTTP and HTTPS from the internet.
- Database security group allows only database traffic from the web or application server security group.

#### Common Confusion Points

Students often confuse security groups with NACLs. Security groups are attached to resources and are stateful. NACLs are attached to subnets and are stateless. This distinction becomes critical again in Week 6.

## 7.5 EC2 Instance Families

The PDF explains that AWS offers many instance families because different workloads stress different resources.

**Main idea:** Compute design is a workload-matching problem. Some workloads need balanced resources. Some need high memory. Some need high CPU. Some need GPUs.

| Family Type | Best For | Reason |
|---|---|---|
| General Purpose | Balanced application servers | Balanced CPU and memory |
| Compute Optimized | CPU-heavy tasks | More processing power per cost unit |
| Memory Optimized | In-memory or cache-heavy workloads | More RAM for large working sets |
| Accelerated / GPU | ML, graphics, parallel compute | Specialized hardware acceleration |

**Exam note:** If asked to choose an instance family, mention the workload pattern first and the family second. That answer style is stronger than simply naming categories.

## 7.6 EC2 Pricing Models

The PDF emphasizes that compute choice is not complete until pricing model is also chosen.

**Main pricing modes to remember:**

- on-demand,
- reserved or savings-oriented commitments,
- spot style excess-capacity pricing where interruption is acceptable.

| Pricing Model | Best Use | Main Trade-Off |
|---|---|---|
| On-Demand | uncertain or short-term usage | highest flexibility, not cheapest |
| Reserved / Savings Plan | stable long-running workloads | lower cost but commitment needed |
| Spot / Interruptible | fault-tolerant batch or flexible workloads | cheapest but interruption risk |

**Why this matters:** Two architectures with the same performance can have very different monthly cost depending on pricing strategy.

## 7.7 Launching and Connecting to an EC2 Instance

The PDF includes practical launch flow and access flow. For theory answers, the useful abstraction is:

1. choose AMI,
2. choose instance type,
3. choose networking and subnet,
4. attach security group,
5. attach or create storage,
6. select key pair,
7. launch,
8. connect using SSH or equivalent secure method.

**Why the sequence matters:** It shows that instance creation is not only a "start server" action. It is a combination of image, hardware, network, storage, and access decisions.

## 7.8 Basic Instance Management

Once launched, an instance goes through an operational lifecycle:

- start,
- stop,
- reboot,
- terminate,
- monitor,
- resize or replace.

**Important exam point:** Stopping is different from terminating. Stopping pauses usage while keeping recoverable instance identity in many cases. Termination destroys the instance lifecycle and is much more final.

## 7.9 EBS: Elastic Block Store

The PDF explains EBS as the disk attached to EC2.

**Definition:** EBS is persistent block storage for EC2 instances.

**How it works:** An EBS volume behaves like a virtual hard disk attached to a server. The instance uses it to store operating system data, application data, or other block-level storage needs.

**Why it is used:**

- persistent storage independent of instance runtime,
- suitable for operating system volumes,
- suitable for database-style block storage,
- can be snapshot-backed for backup.

**Where it is used:**

- boot volumes,
- transactional systems,
- application state stored at block level,
- structured workloads requiring attached disk semantics.

### 7.9.1 Volume Types

The exact catalog is less important than the design principle:

- some volumes optimize for general use,
- some optimize for high performance,
- some optimize for specific IOPS or throughput needs.

**Practical insight:** Database workloads often need stronger performance guarantees than simple web servers.

## 7.10 Snapshots and Backup Logic

An EBS snapshot is a point-in-time backup of a volume stored by AWS.

**Why snapshots matter:**

- disaster recovery,
- cloning environments,
- rollback and restore,
- migration across availability zones or regions depending on design.

**Working intuition:** If EBS is the disk, a snapshot is the recoverable image of that disk at a chosen time.

## 7.11 Attaching and Detaching EBS Volumes

The PDF practical section highlights that storage and compute can be managed separately. This matters because:

- data can survive beyond one instance lifecycle,
- additional storage can be attached for scaling or reconfiguration,
- operational maintenance becomes easier.

## 7.12 Auto Scaling Groups

This is one of the strongest Week 5 topics. The PDF presents Auto Scaling as the production answer to unpredictable demand.

**Definition:** An Auto Scaling Group manages a group of EC2 instances and automatically increases or decreases instance count according to demand or health rules.

**How it works:**

1. define a launch template or launch configuration,
2. set minimum, desired, and maximum capacity,
3. define scaling triggers based on demand or metrics,
4. AWS adds or removes instances accordingly,
5. unhealthy instances can be replaced automatically.

| Metric Rise | Policy Trigger | Instances Added | Capacity Restored |
|---|---|---|---|
| High CPU / Traffic | Scale Out | New EC2 nodes launch | Load spreads |
| Low Demand | Scale In | Extra nodes removed | Cost drops |

Figure 1: Auto Scaling control idea

**Why it is used:**

- absorbs traffic spikes,
- reduces manual intervention,
- improves resilience,
- avoids leaving too many idle instances running.

**Common mistake:** Auto Scaling does not replace good application design. If the application cannot scale horizontally, Auto Scaling alone is not enough.

## 7.13 Elastic Load Balancer

The PDF presents ELB as the traffic distributor that works naturally with multiple EC2 instances.

**Definition:** An Elastic Load Balancer distributes incoming traffic across multiple targets so that no single server becomes overloaded.

**How it works:**

1. clients send requests to the load balancer,
2. the load balancer checks healthy targets,
3. requests are routed across available instances,
4. unhealthy targets are avoided.

**Why it is used:**

- prevents hotspot servers,

- improves availability,
- supports horizontal scaling,
- provides a clean public entry point.

**Real-world use case:** An e-commerce site puts a load balancer in front of multiple web servers. During a sale, new instances are added by Auto Scaling and immediately begin receiving traffic through the same load balancer.

| Service | Main Job | Architecture Benefit |
|---|---|---|
| EC2 | Run workloads | Full OS-level compute control |
| EBS | Persistent block storage | Durable attached disk |
| Auto Scaling Group | Adjust instance count | Demand-responsive capacity |
| Load Balancer | Spread traffic | Availability and distribution |

# 8 Week 6: Cloud Networking Fundamentals and Secure VPC Design

## 8.1 Week 6 Focus

The PDF treats VPC networking as one of the most important practical topics in AWS. Week 6 explains how cloud networking works inside a private logical boundary and how secure architectures are assembled from subnets, routing, gateways, and network controls.

## 8.2 What a VPC Is

**Definition:** A VPC, or Virtual Private Cloud, is a logically isolated private network inside AWS where resources such as instances, databases, and load balancers are placed.

**How it works:**

- You define an IP address range.
- You divide that space into subnets.
- You attach route tables and gateways.
- You place resources into chosen subnets.
- You control access with security groups and NACLs.

**Why it is used:**

- to isolate workloads,
- to control traffic flow,
- to design secure multi-tier systems,
- to separate public and private resources.

**Exam-safe message:** The VPC is the networking boundary that makes secure cloud architecture possible.

## 8.3 CIDR and IP Addressing

The PDF stresses that subnetting and routing make more sense only after CIDR is understood.

**Definition:** CIDR notation expresses an IP range as a base address plus prefix length, such as '/24'.

**Important formula intuition:**

$$\text{Number of IP addresses in a block} = 2^{(32-\text{prefix length})}$$

**Example:**

- '/24' gives $2^8 = 256$ total addresses.
- '/16' gives $2^{16} = 65536$ total addresses.

**AWS practical caveat:** In AWS subnets, not every address is usable because AWS reserves a few addresses in each subnet. For exam answers, the concept matters more than the exact reservation mechanics unless specifically asked.

**Why CIDR matters:**

- address planning,
- subnet sizing,
- avoiding overlap,
- enabling future growth.

## 8.4 Subnets: Public and Private

A subnet is a subdivision of the VPC address space.

### 8.4.1 Public Subnet

A public subnet is a subnet whose route table allows a path to an internet gateway. Resources placed there can have direct internet-facing connectivity if configured appropriately.

### 8.4.2 Private Subnet

A private subnet does not expose resources directly to the public internet. Private resources usually communicate outward through controlled paths such as NAT-based outbound access when required.

**Why the distinction matters:**

- public-facing components belong in controlled public zones,
- sensitive systems such as databases usually belong in private zones,
- separation reduces direct attack surface.

| Internet | Public Subnet | Private Subnet |
|---|---|---|
| Users access web tier | Load balancer / web servers | App / DB / internal services |

Figure 2: High-level public/private subnet idea

## 8.5 Route Tables

**Definition:** A route table tells traffic where to go next.

**How it functions:** Each route table contains rules that map destination ranges to targets such as:

- local VPC routes,
- internet gateways,
- NAT gateways,
- peering connections.

**Why it is important:** Subnet exposure is determined not only by where the resource is placed, but also by the route table attached to that subnet.

## 8.6 Internet Gateway and NAT Gateway

### 8.6.1 Internet Gateway

An internet gateway gives public-facing traffic connectivity between a VPC and the internet.

### 8.6.2 NAT Gateway

A NAT gateway allows instances in private subnets to initiate outbound internet access without accepting direct inbound traffic from the internet.

| Component | Main Role | Typical Use |
|---|---|---|
| Internet Gateway | Public internet path | Web or internet-facing entry points |
| NAT Gateway | Outbound internet for private resources | Private instances downloading updates or dependencies |

**Why the pair matters:** This is the core mechanism behind public/private subnet design in AWS.

## 8.7 Security Groups vs NACLs

This is one of the most exam-friendly comparison topics in the PDF.

| Aspect | Security Group | Network ACL |
|---|---|---|
| Applied to | resource / instance level | subnet level |
| State model | stateful | stateless |
| Rule style | allow-focused | allow and deny style filtering |
| Typical purpose | fine-grained resource protection | coarse subnet boundary filtering |

**Interpretation:** Security groups protect the door of the resource. NACLs protect the boundary of the subnet.

## 8.8 Connecting Resources Across Subnets

Resources in different subnets can still communicate if:

- routing permits the path,
- security groups permit the traffic,
- NACLs do not block it.

**Why students get confused:** They often assume different subnets cannot communicate automatically. Inside a VPC, subnets can communicate if control rules allow it.

## 8.9 Public IPs vs Elastic IPs

**Public IP:** A public address assigned for internet reachability.

**Elastic IP:** A static public IP reserved for stable mapping.

**Why the distinction matters:** If a public identity must remain stable across restarts or replacements, an elastic IP may be more suitable than a transient public IP.

## 8.10 VPC Peering

VPC peering is a direct network connection between two VPCs.

**Why it is used:**

- connect related network spaces,
- allow resource access across isolated VPCs,
- keep traffic private without public internet routing.

**Caveat:** It is simple but not always the best choice at very large scale. For exam answers, focus on its purpose as a private network-to-network link.

## 8.11 Two-Tier Architecture Design

The PDF includes a two-tier design exercise because it integrates multiple networking concepts into one architecture.

**Typical two-tier design:**

- tier 1: web tier in a public subnet,
- tier 2: database tier in a private subnet.

**Workflow:**

1. User traffic reaches the public-facing layer.
2. The web tier processes requests.
3. The web tier communicates with the database tier privately.
4. The database is not directly reachable from the internet.

| Users | Public Entry | Web Tier | Private DB Tier |
|---|---|---|---|
| Internet requests | Load balancer / web subnet | App logic | Database only from app tier |

Figure 3: Two-tier VPC design mental model

## 8.12 VPC Security Best Practices

The PDF frames VPC security as a design discipline rather than one feature.

**Best practices:**

- use private subnets for internal resources,
- open only required ports,

- separate web, app, and database tiers,
- use least privilege in both IAM and network controls,
- avoid public exposure unless there is a clear business need,
- review routing and filtering together rather than in isolation.

**Exam Focus**

Network questions in exams are rarely only about definitions. The stronger answer explains traffic path, subnet role, gateway role, and security boundary together.

# 9 Week 7: Containers, Docker, Registries, and Kubernetes Basics

## 9.1 Week 7 Focus

The PDF introduces containers as the foundation of modern cloud-native delivery. This week explains why containers became popular, how Docker works, why registries matter, and how Kubernetes solves multi-container orchestration.

## 9.2 What Containers Are

**Definition:** A container is a lightweight package that bundles application code with its dependencies and runtime requirements.

**How it works:** Instead of virtualizing full hardware and full operating systems, containers share the host OS kernel while isolating application processes and environments.

**Why it is used:**

- consistency across development and production,
- portability,
- fast startup,
- efficient resource usage,
- easier packaging of services.

**Where it is used:**

- microservices,
- API backends,
- CI/CD pipelines,

- reproducible local development,
- scalable platform deployments.

## 9.3 Containers vs Virtual Machines

This is one of the most important conceptual comparisons in Week 7.

| Aspect | Virtual Machines | Containers |
|---|---|---|
| Isolation unit | full OS instance | application process environment |
| Weight | heavier | lighter |
| Startup time | slower | faster |
| Portability | good, but heavier image footprint | very high for app packaging |
| Resource efficiency | lower than containers | higher than VMs |

**Interpretation:** VMs are stronger environment isolation units. Containers are faster and lighter for application packaging and scaling.

## 9.4 Why Containers Matter

The PDF emphasizes three benefits strongly:

- consistency,
- portability,
- operational speed.

**Consistency:** The same container image can run in development, testing, and production.

**Portability:** If the container runtime exists, the workload can move more easily across environments.

**Speed:** Containers start quickly and support rapid deployment and replacement.

**Real-world use case:** If a team needs to release a hotfix during a traffic spike, a corrected image can be built, tagged, and deployed across the cluster quickly.

## 9.5 Docker Internals Overview

Docker is the most common beginner entry point into container tooling.

**Useful mental model from the PDF:**

- image = packaged blueprint,
- container = running instance of that blueprint,
- Dockerfile = recipe for building the image,
- registry = storage place for images.

**How Docker works at a high level:**

1. Read build instructions from a Dockerfile.
2. Build an image in layers.
3. Store the image locally or in a registry.
4. Run a container from the image.
5. Stop and remove the container when needed.

## 9.6 Dockerfile

**Definition:** A Dockerfile is a text file containing instructions that define how an image should be built.

**Why it matters:**

- build becomes reproducible,
- image creation is versionable,
- teams can rebuild the same environment consistently.

**Common logic in a Dockerfile:**

- choose base image,
- copy application files,
- install dependencies,
- set working directory,
- expose service port,
- define startup command.

## 9.7 Container Lifecycle and Image Versioning

This topic is both PDF-backed and sample-paper-backed, so it deserves strong treatment.

1. Write build instructions.
2. Build the image.
3. Tag the image with a version.
4. Push the image to a registry.
5. Pull the image where deployment is needed.
6. Run one or more containers from the image.
7. Stop, replace, or terminate containers during updates.

**Why versioning matters:**

- reproducibility,
- rollback,
- deployment traceability,
- environment consistency.

## 9.8 Container Registry and ECR

**Registry definition:** A container registry stores container images so they can be pulled into deployment environments.

**Push and pull logic:**

- push uploads an image to the registry,
- pull downloads an image from the registry to a runtime environment.

**Why registries matter:**

- team sharing,
- deployment repeatability,
- central image management,
- controlled software supply path.

**ECR:** Amazon ECR is AWS's managed container registry service. Its role is to store and manage container images for AWS-based deployment workflows.

## 9.9 Kubernetes and Orchestration Basics

The PDF introduces Kubernetes because running one container manually is not the same as operating many containers reliably at scale.

**Definition:** Kubernetes is an orchestration platform for deploying, scaling, and managing containerized applications.

**Why orchestration is needed:**

- many containers must be scheduled across nodes,
- failed containers must be replaced,
- scaling should be automated,
- updates should happen safely,
- service connectivity must be managed.

**Core idea:** Docker solves packaging. Kubernetes solves large-scale operational management of those packages.

## 9.10 Kubernetes Architecture Overview

The PDF uses a simple architecture explanation rather than deep Kubernetes internals. The exam-ready abstraction is:

- control plane decides desired state,
- worker nodes run application containers,
- the platform watches health and keeps workloads aligned to desired state.

**How it functions conceptually:**

1. desired application state is declared,
2. Kubernetes schedules containers on suitable nodes,
3. failed instances are replaced,
4. scaling changes replica count,
5. service exposure keeps traffic flowing to healthy workloads.

## 9.11 How Kubernetes Manages Workloads

The PDF presents workload management as:

- keep desired count running,
- reschedule on failure,
- distribute traffic,
- support rolling updates.

**Practical example:** If a deployment should have 5 replicas and 1 fails, Kubernetes notices the gap and creates a replacement to return to desired state.

| Image | Container Replicas | Kubernetes Watches | Desired State Restored |
|---|---|---|---|
| Versioned package | Many running units | Health / count / placement | Replacement or scale action |

Figure 4: Kubernetes workload control idea

# 10 Week 8: Storage Services, S3, EBS, EFS, and Data Placement

## 10.1 Week 8 Focus

Week 8 in the PDF concentrates on storage design. The important theme is that storage is not one thing. Different workload needs lead to different storage choices.

## 10.2 Types of Cloud Storage

The PDF teaches storage first by category, then by AWS service.

**Main storage types:**

- object storage,
- block storage,
- file storage.

| Storage Type | Best Fit | Typical AWS Example |
|---|---|---|
| Object Storage | files, media, logs, backups, static content | S3 |
| Block Storage | attached server disks, OS and databases | EBS |
| File Storage | shared file access across systems | EFS |

**Why this matters:** Architects do not choose storage randomly. They choose based on access pattern, sharing need, performance shape, and cost profile.

## 10.3 S3, EBS, and EFS Overview

### 10.3.1 S3

S3 is object storage. It is used for large-scale durable storage of unstructured data.

### 10.3.2 EBS

EBS is persistent block storage attached to EC2.

### 10.3.3 EFS

EFS is managed file storage designed for shared access across multiple systems.

## 10.4 Choosing the Right Storage Type

The PDF frames storage choice as a design decision.

| Need | Best Choice | Why |
|---|---|---|
| Store documents, images, logs | S3 | scalable object storage with durability |
| Attach disk to one virtual server | EBS | block semantics for server workloads |
| Share the same file system across machines | EFS | managed multi-client file access |

## 10.5 S3 Storage Classes

The PDF emphasizes that not all data should live in the same cost-access tier.

**Core logic:** Frequent access data belongs in fast commonly used classes. Infrequent access or archival data should move to lower-cost colder tiers.

**Examples of class intent:**

- standard for active data,
- infrequent access for less-used but still available data,
- archival classes such as glacier-style tiers for cold retention.

**Why it is used:** It aligns storage cost with access behavior.

## 10.6 S3 Optimization Features

The PDF highlights versioning, lifecycle rules, and cost optimization.

### 10.6.1 Versioning

Versioning keeps multiple versions of objects.

**Why useful:**

- recovery from accidental overwrite,
- recovery from accidental deletion,
- auditability of object changes.

### 10.6.2 Lifecycle Rules

Lifecycle rules automatically transition or delete data based on policy.

**Why useful:**

- reduces manual cleanup,
- moves cold data to cheaper classes,
- avoids cost sprawl.

## 10.7 Bucket Policies and IAM Permissions

The PDF treats S3 access control as one of the most misunderstood topics.

**Main access control layers:**

- IAM permissions,
- bucket policies,
- block public access settings.

**Interpretation:** IAM usually controls who can do what from the identity side. Bucket policies control access at the bucket boundary.

## 10.8 S3 Security Settings

Many public cloud incidents involve accidental storage exposure. The PDF explains S3 Block Public Access precisely because this is a real operational risk.

**Key idea:** Public exposure should be intentional, not accidental.

**Why Block Public Access matters:**

- prevents easy misconfiguration,
- reduces accidental data leakage,
- supports safer default posture.

## 10.9 S3 Encryption Options

The PDF emphasizes that stored data should be protected, not just available.

**Why encryption is used:**

- protects stored data from unauthorized access,
- supports compliance expectations,
- reduces exposure severity if storage is mishandled.

**Exam-safe answer:** State that S3 encryption protects data at rest and is often combined with access control, not used as a substitute for it.

## 10.10 S3 Static Website Hosting

The PDF includes S3 website hosting to show that S3 is not only for passive storage.

**Use case:** Host static HTML, CSS, JS, and media assets without running a full compute server.

**Why useful:**

- simple deployment,
- low operational burden,
- cost efficiency for static content.

## 10.11 EFS Shared File Storage

The PDF contrasts EFS with EBS.

**Why EFS exists:** Some workloads need a shared file system accessible by multiple compute nodes. That is different from attaching one block device to one server.

**Where it is used:**

- shared content repositories,
- collaborative file access across instances,
- workloads expecting normal file system semantics at shared scale.

| Service | Access Pattern | Best Use |
|---|---|---|
| S3 | API-based object access | media, logs, backups, static sites |
| EBS | attached block device | EC2 boot and block-level workloads |
| EFS | shared file system access | multi-instance shared file workloads |

# 11 Week 9: Databases, RDS, Aurora, DynamoDB, and Access-Pattern Thinking

## 11.1 Week 9 Focus

The PDF treats database choice as an architectural decision driven by access patterns. The main objective is not memorizing brand names. It is understanding when relational structure is necessary and when NoSQL scale and flexibility are more suitable.

## 11.2 Relational vs NoSQL

### 11.2.1 Relational Databases

Relational databases organize data into structured tables with relationships.

**Best for:**

- strong consistency,
- structured querying with SQL,
- transactions,
- workloads where data relationships matter deeply.

### 11.2.2 NoSQL Databases

NoSQL systems use more flexible data models and are often designed for scale, speed, or specific access patterns.

**Best for:**

- high-scale key-based access,
- flexible schema needs,
- workloads where distribution and low latency matter more than complex joins.

| Aspect | Relational | NoSQL |
|---|---|---|
| Data model | structured tables with relationships | flexible model such as key-value or document |
| Query style | rich relational querying | access-pattern-driven queries |
| Best for | integrity and relationship-heavy systems | scale and simple high-speed access |

**Real-world examples from the PDF framing:**

- relational: orders, payments, user accounts, inventory integrity,
- NoSQL: session storage, carts, leaderboards, real-time profiles, chat or event-heavy systems.

## 11.3 Managed vs Self-Managed Databases

The PDF pushes this as a practical design choice.

**Managed database:** The provider handles operational tasks such as patching, backups, and failover support.

**Self-managed database:** The team handles installation, patching, tuning, recovery, and operation directly.

**Why managed databases matter:**

- less operational burden,
- quicker setup,
- easier recovery features,
- better fit for teams focusing on application logic.

## 11.4 AWS Database Landscape

The PDF highlights RDS, Aurora, and DynamoDB as major database choices in AWS.

| Service | Model | Best Fit |
|---|---|---|
| RDS | managed relational database | SQL-based operational workloads |
| Aurora | cloud-optimized relational database | high-performance managed relational needs |
| DynamoDB | managed NoSQL key-value / document style | scale-out low-latency access patterns |

## 11.5 RDS Engines and Features

The PDF explains that multiple relational engines exist because not every enterprise prefers the same database ecosystem.

**Why engines differ:**

- ecosystem compatibility,
- licensing,
- tooling preference,
- workload history,
- enterprise policy.

**Exam-safe message:** RDS is a managed relational database platform that supports multiple engines so teams can keep SQL-style data models while reducing operational burden.

## 11.6 RDS Resilience and Backups

This is one of the most important reliability topics in the database module.

**Main resilience mechanisms from the PDF:**

- multi-AZ deployment,
- automated backups,
- snapshots,
- failover support.

**Why they matter:**

- reduce downtime risk,
- improve recoverability,
- support business continuity,
- align database operation with uptime goals.

## 11.7 Read Replicas, Scaling, and Failover

**Read replicas:** Extra read copies of the database used to offload read traffic.

**Why useful:** Read-heavy systems often fail first on read contention rather than pure data size.

**Failover:** If the primary path fails, service can shift to a healthy replacement according to architecture design.

**Scaling insight:** Database scaling is harder than simple stateless web scaling. That is why read replicas, backups, and managed resilience features matter so much.

## 11.8 DynamoDB Concepts

The PDF explains DynamoDB through the key-value model.

**Definition:** DynamoDB is a managed NoSQL database service designed for very fast key-based access and large-scale distributed workloads.

**How it works conceptually:** Each item is identified by a key. When access patterns are well designed around that key structure, retrieval becomes extremely fast.

**Why it is used:**

- no server management,
- high scale,
- low latency,
- automatic operational features.

## 11.9 DynamoDB Capacity and Pricing

The PDF emphasizes that performance and cost are linked to read and write behavior.

**Core idea:** In DynamoDB, access pattern design affects both performance and billing. This is why bad data modeling creates cost and latency pain.

**Exam-safe phrasing:** DynamoDB capacity planning is closely tied to expected read and write intensity, unlike traditional databases where teams often think first in terms of server sizing.

## 11.10 Where DynamoDB Fits

The PDF gives a strong fit-vs-misfit teaching style.

**Good fit:**

- session stores,
- shopping carts,
- user profile lookups,
- high-scale key-value access,
- low-latency event-heavy systems.

**Weak fit:**

- deeply relational workloads,
- complex ad hoc relational joins,
- systems whose data model depends on rich SQL relationships.

**Exam Focus**

In database answers, start from access pattern and correctness requirement. That matches the PDF and produces stronger reasoning than service-name memorization.

# 12 Week 10: Serverless Computing, Lambda, and Event-Driven Design

## 12.1 Week 10 Focus

The PDF presents serverless as a major mindset shift. Instead of managing long-running servers, teams write logic that runs in response to events.

## 12.2 What Serverless Means

**Definition:** Serverless computing means the cloud provider manages the underlying infrastructure while code executes only when triggered by events.

**Important correction:** Serverless does not mean no servers exist. It means the user does not manage them directly.

**Why it is used:**

- no server management,
- automatic scaling,
- pay-per-use cost model,
- strong fit for unpredictable or idle-heavy workloads.

**Where it is used:**

- automation,
- file processing,
- lightweight APIs,
- notifications,
- scheduled jobs,
- event-driven data workflows.

## 12.3 AWS Lambda Fundamentals

Lambda is AWS's main serverless compute service.

**Definition:** A Lambda function is a small unit of code that runs when triggered by an event.

**Main internal ideas from the PDF:**

- handler = entry point,
- execution environment = runtime plus code plus dependencies,
- warm starts may reuse an environment,
- cold starts create new environments,
- functions are naturally stateless between independent invocations.

**Execution limit:** The PDF clearly states Lambda is intended for short event-driven work and has a maximum execution time limit. That is why it is not a replacement for every always-on workload.

## 12.4 Lambda Pricing Model and Cost Advantages

**Main pricing factors from the PDF:**

- number of requests,
- execution duration,
- memory allocation.

**Cost intuition:** You pay when code runs, not for idle server uptime.

**Practical insight from the PDF:** More memory can sometimes reduce total cost because extra CPU power helps the function finish faster.

## 12.5 Lambda Configuration and IAM Role Usage

The PDF practical section strongly connects configuration and permissions.

**Configuration controls:**

- runtime,
- architecture choice,
- memory,
- timeout,
- environment variables,

- concurrency.

**Permission model:** Lambda itself has no permissions by default. It uses an IAM role to access other AWS services.

**Critical distinction:**

- IAM user = human or long-lived identity,
- IAM role = temporary delegated permission set for services or workloads.

## 12.6 Concurrency, Memory, and Timeout

The PDF gives a very good three-part mental model:

- memory controls power per execution,
- timeout controls how long one execution may run,
- concurrency controls how many executions can run at once.

**Why this matters:** Many real Lambda failures are configuration problems, not code syntax problems.

## 12.7 Monitoring Lambda Functions

The PDF shows Lambda monitoring using CloudWatch.

**What teams observe:**

- invocation count,
- errors,
- duration,
- logs,
- throttling behavior.

**Why monitoring is needed:**

- detect failed executions,
- catch rising latency,
- troubleshoot permission or payload problems,
- understand cost behavior.

## 12.8 Event Sources

The PDF frames event sources as the real heart of serverless.

**Definition:** An event source is the service or action that triggers a Lambda function.

**Examples:**

- API Gateway request,
- S3 object upload,
- scheduled event,
- database stream or service event.

## 12.9 S3-Triggered Lambda

**Workflow:**

1. a file is uploaded to S3,
2. S3 emits an event,
3. Lambda executes processing logic,
4. result is stored or forwarded elsewhere.

**Why useful:** This pattern removes the need for continuously running servers waiting for file events.

## 12.10 API Gateway and Lambda

**Workflow:**

1. client sends HTTP request to API Gateway,
2. API Gateway triggers Lambda,
3. Lambda runs business logic,
4. response is returned through API Gateway.

**Why useful:** It creates a serverless API backend with automatic scaling and no VM management.

| Client Request | API Gateway | Lambda Function | Response |
|---|---|---|---|
| HTTP call | routes event | runs logic | returns result |

Figure 5: API Gateway + Lambda serverless API flow

## 12.11 End-to-End Serverless Workflow

The PDF emphasizes event-driven thinking:

- something happens,
- code runs,
- output is produced,
- no server is kept waiting all the time.

## 12.12 Statelessness in Serverless

This is one of the most important conceptual topics in the module.

**Definition:** Statelessness means each execution should not depend on local memory from previous executions.

**Why it matters:**

- supports scaling,
- supports independent execution,
- avoids fragile assumptions about reused runtime memory.

**Implication:** If state is needed, it should be stored externally in systems such as S3, DynamoDB, or databases.

## 12.13 Limitations and Best Practices

The PDF clearly warns that serverless is powerful but not universal.

**Limitations:**

- execution time limits,
- cold start impact,
- not ideal for long-running jobs,
- not ideal for stateful always-on application patterns,
- not always cheapest for constant heavy traffic.

**Best practices:**

- keep functions focused,
- keep them stateless,
- use least privilege IAM roles,
- monitor execution and failures,
- choose serverless only when the workload pattern fits.

# 13 Week 11: Monitoring, Auditing, and Cost Optimization

## 13.1 Week 11 Focus

This module in the PDF shifts from building systems to operating them responsibly. The main themes are visibility, auditing, and cost control.

## 13.2 CloudWatch Metrics

**Definition:** Metrics are numeric measurements collected over time that describe how resources or applications behave.

**Examples:**

- CPU usage,
- request count,
- latency,
- error count,
- invocation duration.

**Why metrics are used:**

- trend analysis,
- capacity observation,
- anomaly detection,
- scaling and alert input.

## 13.3 CloudWatch Alarms

Metrics alone show numbers. Alarms add action and notification.

**Definition:** A CloudWatch alarm watches a metric and reacts when a threshold or condition is met.

**Why alarms matter:**

- bring attention to critical issues,
- enable fast incident response,
- support automation and escalation.

## 13.4 CloudWatch Logs

**Definition:** CloudWatch Logs is AWS's centralized logging service for collecting and examining system and application logs.

**Why logs matter:**

- metrics tell you that something is wrong,
- logs help explain what happened,
- troubleshooting often depends on logs, not metrics alone.

## 13.5 CloudTrail

**Definition:** CloudTrail records account activity and API actions in AWS.

**Core question it answers:** *Who did what and when?*

**Why CloudTrail matters:**

- auditing,
- governance,
- security visibility,
- compliance evidence.

## 13.6 CloudTrail Integrations

The PDF notes that CloudTrail becomes more useful when integrated with storage and monitoring services.

**Why integrations matter:**

- event records can be stored safely,
- alerts can be created from suspicious actions,
- governance becomes operational rather than passive.

## 13.7 Billing Dashboard and AWS Pricing Visibility

**Main insight:** Cloud cost is dynamic. Unlike fixed data center spending, cloud bills move with usage and bad decisions can become visible very quickly.

**Why billing visibility matters:**

- early detection of unusual spend,
- better forecasting,

- accountability across teams,
- informed optimization.

## 13.8 AWS Budgets and Cost Explorer

The PDF treats these as core operational tools.

**AWS Budgets:** Used to set spending expectations and receive warning signals.

**Cost Explorer:** Used to analyze where money is going and how usage patterns evolve.

**Why both matter together:**

- budgets warn,
- explorer explains.

## 13.9 Cost Tagging

**Definition:** Tags are labels attached to cloud resources to help organize and attribute usage.

**Why tags matter:**

- cost allocation,
- ownership tracking,
- environment grouping,
- operations and governance clarity.

## 13.10 Rightsizing and Storage Optimization

**Definition:** Rightsizing means choosing resources that match actual workload needs rather than guessed or excessive capacity.

**Why it matters:**

- over-provisioning wastes money,
- under-provisioning damages performance,
- optimization should be continuous, not one-time.

## 13.11 Reserved Instances and Savings Logic

The PDF teaches that stable workloads can be made cheaper through longer-term pricing commitments.

**Design insight:** Use flexible on-demand pricing for uncertainty. Use commitment-based pricing when workload behavior is steady enough to justify lower cost through commitment.

## 13.12 Cost-Efficient Architectures

The PDF connects cost optimization back to design, not only billing tools.

**Examples:**

- autoscaling instead of always-on overcapacity,
- serverless for event-driven bursty workloads,
- lifecycle policies for storage,
- rightsizing compute and storage,
- better purchasing models for stable workloads.

| Operational Need | Service / Technique | Why It Helps |
|---|---|---|
| Observe behavior | Metrics and logs | visibility into health and usage |
| Detect incidents | Alarms | early response |
| Audit actions | CloudTrail | accountability and governance |
| Control spend | Budgets and Cost Explorer | proactive cost management |
| Reduce waste | Rightsizing / autoscaling / lifecycle policies | cost-efficient architecture |

# 14 Week 12: DevOps in the Cloud, Version Control, CI/CD, and Deployment Strategies

## 14.1 Week 12 Focus

The PDF moves from infrastructure operation into software delivery. The key idea is that cloud value becomes much greater when delivery is automated, repeatable, and collaborative.

## 14.2 What DevOps Is

**Definition:** DevOps is a set of practices and ways of working that brings development and operations together to improve software delivery speed, quality, and reliability.

**What DevOps is not:**

- not only a tool,
- not only a job title,

- not only writing scripts.

**Why it matters in cloud:** Cloud provides rapid infrastructure. DevOps provides rapid, safer delivery on top of that infrastructure.

## 14.3 Why DevOps Matters

The PDF stresses speed, consistency, and repeatability.

**Main benefits:**

- faster release cycles,
- lower manual error,
- better collaboration,
- easier rollback and repeatability,
- more reliable delivery processes.

## 14.4 Version Control

**Definition:** Version control tracks changes to code and allows teams to collaborate safely over time.

**Why it is used:**

- history,
- collaboration,
- rollback,
- branch-based experimentation,
- better code review and accountability.

## 14.5 Git Essentials

The PDF keeps Git discussion practical. The concepts that matter most are:

- repository,
- commit,
- branch,
- merge,
- push and pull behavior.

**Why Git matters in cloud and DevOps:** Automated delivery pipelines usually begin from source control events.

## 14.6 GitHub and Repository Workflows

The PDF explains GitHub as the collaboration platform built around repository workflows.

**Why GitHub matters:**

- pull request based review,
- collaboration across teams,
- integration with CI/CD,
- repository-based traceability.

## 14.7 Components of a CI/CD Pipeline

This is one of the most exam-relevant topics because it appears directly in the sample paper.

**Definition:** A CI/CD pipeline is an automated flow that builds, tests, and delivers software changes from source code toward deployment.

**Typical stages:**

1. code commit,
2. build,
3. test,
4. package,
5. deploy to staging,
6. approve or automatically release to production.

| Commit | Build | Test | Package | Deploy |
|---|---|---|---|---|
| Change enters system | artifact created | quality checked | release unit prepared | staged or released |

Figure 6: Basic CI/CD pipeline flow

**Why CI/CD matters:**

- faster iteration,
- fewer manual deployment mistakes,
- early detection of broken code,
- higher repeatability,
- better delivery confidence.

## 14.8 CI/CD Tools Overview

The PDF explains tools as automation engines that implement build, test, and deployment workflows.

**Exam-safe wording:** CI/CD tools automate the path from code change to validated deployment so teams do not depend on manual repetitive release work.

## 14.9 Pipeline Triggers and Testing

The PDF explains that triggers and automated tests are the backbone of trusted pipelines.

**Common triggers:**

- code push,
- pull request,
- merge to a main branch,
- schedule,
- manual approval trigger.

**Why testing matters:** If tests fail, broken code should stop before production. This is why the pipeline is not only a delivery tool but a quality gate.

## 14.10 Deployment Strategies

The PDF covers blue-green, rolling, and canary deployments.

### 14.10.1 Blue-Green Deployment

Two environments are maintained. Traffic is shifted from the old environment to the new environment after validation.

**Strength:** Fast rollback by switching traffic back.

### 14.10.2 Rolling Deployment

Instances are updated gradually instead of all at once.

**Strength:** Smoother change with lower simultaneous replacement risk.

### 14.10.3 Canary Deployment

A small subset of users or traffic is sent to the new version first.

**Strength:** Very controlled risk exposure before wider rollout.

| Strategy | Main Idea | Best When |
|---|---|---|
| Blue-Green | switch traffic between old and new environments | rollback speed is very important |
| Rolling | update gradually across running fleet | service should remain live during gradual replacement |
| Canary | test new version on small subset first | risk must be minimized before full rollout |

# 15 Week 13: Infrastructure as Code and Terraform

## 15.1 Week 13 Focus

The PDF ends by moving from manual cloud usage to programmable infrastructure management. This is the bridge from cloud understanding to real delivery engineering.

## 15.2 What Infrastructure as Code Means

**Definition:** Infrastructure as Code, or IaC, means defining infrastructure using code or configuration files rather than creating it manually through a console.

**Why it is used:**

- repeatability,
- consistency,
- version control,
- easier review,
- easier automation,
- reduced configuration drift.

**Simple analogy from the PDF style:** Manual infrastructure is like cooking without a recipe. IaC is like using a written recipe that can be repeated reliably.

## 15.3 Declarative vs Imperative IaC

### 15.3.1 Declarative

You describe the final desired state. The tool figures out how to reach it.

### 15.3.2 Imperative

You explicitly describe the sequence of steps to be executed.

| Approach | Main Style | Example Thinking |
|---|---|---|
| Declarative | describe desired end state | "I want two servers and one network" |
| Imperative | describe exact action sequence | "create this, then attach that, then configure this" |

**Why the distinction matters:** Terraform follows a declarative model. That is central to how it plans and applies changes.

## 15.4 IaC Tools Overview

The PDF mentions multiple tools to show that IaC is a category, not one product.

**Exam-safe framing:**

- Terraform is widely used and multi-provider oriented,
- CloudFormation is AWS-native,
- Pulumi uses general-purpose programming languages for IaC logic.

## 15.5 Terraform Workflow

This is one of the strongest Week 13 exam topics.

**Core lifecycle:**

1. write configuration,
2. initialize providers,
3. plan changes,
4. apply changes,
5. manage resulting state.

| Write Config | Init / Plan | Apply | State Updated |
|---|---|---|---|
| Desired infra declared | difference is calculated | APIs are called | result is tracked |

Figure 7: Terraform workflow mental model

## 15.6 Providers, Resources, and Modules

The PDF treats these as Terraform's three most important structural concepts.

### 15.6.1 Provider

The provider is the translator between Terraform and the target platform such as AWS.

### 15.6.2 Resource

A resource is a managed infrastructure object such as an EC2 instance or VPC.

### 15.6.3 Module

A module is a reusable group of Terraform configuration that packages related infrastructure logic.

| Concept | Meaning |
|---|---|
| Provider | plugin that lets Terraform talk to a platform such as AWS |
| Resource | actual infrastructure object under management |
| Module | reusable bundle of Terraform configuration logic |

## 15.7 Terraform State

The state file is one of the most critical concepts in Terraform.

**Definition:** Terraform state is the record Terraform keeps of managed infrastructure and its known current condition.

**Why it matters:**

- Terraform compares desired state with known actual state,
- collaboration depends on reliable state handling,
- careless state management can cause conflicting changes.

**Important PDF insight:** State should be protected carefully because it is central to automation trust and coordination.

## 15.8 Understanding `.tf` Files

Terraform reads configuration from files ending in `.tf`.

**Why this matters:** Infrastructure definition becomes code-like, reviewable, and structured.

## 15.9 Variables and Outputs

The PDF presents variables and outputs as key concepts that make configuration reusable.

**Variables:** Let the same configuration behave differently for development, testing, and production.

**Outputs:** Expose useful resulting values such as IPs or resource identifiers after provisioning.

## 15.10 Terraform and the AWS Provider

The PDF gives a clean explanation here: Terraform cannot create AWS resources on its own. It needs the AWS provider as a translation layer into AWS API calls.

**Workflow intuition:**

1. Terraform reads desired configuration.
2. The AWS provider translates that intent.
3. AWS APIs perform the actual resource operations.
4. Terraform records the result in state.

## 15.11 Terraform Apply Execution Flow

The PDF explains that ‘apply‘ is more than just “create resources.”

**What happens conceptually:**

1. configuration is parsed,
2. current state is checked,
3. differences are calculated,
4. change plan is prepared,
5. API calls are executed,
6. state is updated.

**Why this matters:** Understanding the flow helps teams trust automation and debug failures.

## 15.12 Why IaC Matters for Exams and Practice

IaC is not just a DevOps bonus topic. It connects directly to:

- repeatability,

- auditability,
- automation,
- safer infrastructure changes,
- environment consistency.

# 16 Exam Pattern and Answer-Writing Strategy

The model paper in `Cloud Services & Platforms.docx` shows a very clear question style. It prefers answers that combine concept, comparison, architecture logic, and practical use context.

## 16.1 What the Teacher Appears to Prefer

- compare-and-contrast answers with business reasoning,
- architecture and workflow explanations,
- short theory with clear applied meaning,
- operational or deployment life-cycle questions,
- use-case driven decisions rather than pure memorization,
- answers that explain why one option is more suitable than another.

## 16.2 Strong Answer Pattern 1: Definition + Working + Use Case

Use this for concepts such as:

- Lambda,
- IAM role,
- VPC,
- object storage,
- DynamoDB.

### 16.3 Strong Answer Pattern 2: Comparison Table + Recommendation

Use this for:

- horizontal vs vertical scaling,
- security groups vs NACLs,
- relational vs NoSQL,
- blue-green vs rolling vs canary,
- declarative vs imperative IaC.

### 16.4 Strong Answer Pattern 3: Architecture Workflow

Use this for:

- secure VPC design,
- API Gateway plus Lambda flow,
- CI/CD pipeline explanation,
- Terraform apply lifecycle,
- Auto Scaling plus load balancing workflow.

### 16.5 Strong Answer Pattern 4: Scenario Decision

When the question asks "which is better" or "what would you choose," answer in this order:

1. state the business or workload requirement,
2. choose the concept or service,
3. justify using architecture logic,
4. mention one trade-off.

## 17 Possible Questions

### 17.1 Short Concept Questions

1. Define cloud computing and state any two key traits.
2. What is the difference between scalability and elasticity?

3. Define high availability.
4. What is meant by multi-tenancy?
5. What is the purpose of a security group?
6. What is an AMI?
7. Define EBS snapshot.
8. What is the function of a route table in a VPC?
9. Distinguish between public and private subnets.
10. What is a container image?
11. What is a Dockerfile?
12. What is the role of a container registry?
13. Define object storage.
14. What is the difference between block storage and file storage?
15. What is a read replica?
16. What is the basic idea of DynamoDB?
17. What is meant by serverless computing?
18. What is the role of an IAM role in Lambda?
19. What does CloudTrail record?
20. What is the purpose of cost tagging?
21. What is version control?
22. State the major stages of a CI/CD pipeline.
23. What is infrastructure as code?
24. What is Terraform state?
25. What is meant by a provider in Terraform?

## 17.2 Medium Descriptive Questions

1. Explain why cloud computing is considered a utility-style computing model.
2. Explain the shared responsibility model with a simple cloud security example.
3. Explain the difference between horizontal scaling and vertical scaling with suitable cloud use cases.
4. Explain how an Auto Scaling Group works in AWS.

5. Explain the role of Elastic Load Balancing in compute architecture.
6. Explain CIDR and why it matters in VPC design.
7. Explain how internet gateway and NAT gateway support different subnet designs.
8. Explain why containers became important in cloud-native engineering.
9. Explain how Docker images, registries, and running containers are connected.
10. Explain the role of Kubernetes in managing container workloads.
11. Explain how S3 versioning and lifecycle rules support cost control and recoverability.
12. Explain the differences between S3, EBS, and EFS.
13. Explain the difference between relational databases and NoSQL databases using access-pattern thinking.
14. Explain how RDS improves reliability through backups and multi-AZ design.
15. Explain where DynamoDB fits best and where it does not.
16. Explain how Lambda functions are triggered and executed.
17. Explain why statelessness is important in serverless applications.
18. Explain the relationship between CloudWatch metrics, alarms, and logs.
19. Explain how AWS Budgets and Cost Explorer improve financial control.
20. Explain why DevOps becomes more powerful when combined with cloud infrastructure.
21. Explain blue-green deployment and rolling deployment with practical differences.
22. Explain why Terraform is called declarative.

## 17.3 Comparison-Based Questions

1. Compare IaaS, PaaS, and SaaS in terms of responsibility and control.
2. Compare public cloud, private cloud, hybrid cloud, and multi-cloud.
3. Compare high availability and fault tolerance.
4. Compare security groups and network ACLs.
5. Compare public IP and elastic IP.
6. Compare containers and virtual machines.
7. Compare S3, EBS, and EFS.
8. Compare relational databases and NoSQL databases.

9. Compare RDS and DynamoDB from a workload-fit perspective.
10. Compare Lambda and EC2 for application execution.
11. Compare metrics, logs, and audit trails.
12. Compare blue-green, rolling, and canary deployment strategies.
13. Compare declarative IaC and imperative IaC.
14. Compare Terraform, CloudFormation, and Pulumi at a high level.

## 17.4 Architecture and Workflow Questions

1. Design a simple two-tier VPC architecture for a web application and explain traffic flow between components.
2. Explain how security groups, private subnets, and a bastion or controlled entry path can be used to secure backend systems.
3. Explain the full lifecycle of an EC2 workload from launch template to scaling and traffic distribution.
4. Explain the lifecycle of a container from Dockerfile to registry to runtime.
5. Explain how API Gateway and Lambda together implement a serverless API.
6. Explain an S3-triggered Lambda workflow for file processing.
7. Explain the workflow of a CI/CD pipeline from commit to deployment.
8. Explain how Terraform init, plan, and apply work together.
9. Explain how CloudTrail plus CloudWatch supports auditing and operational visibility.
10. Explain how rightsizing, autoscaling, and serverless design can be combined to reduce cost.

## 17.5 Application and Scenario Questions

1. A startup wants full OS control for a legacy application but also wants cloud flexibility. Which compute model would you choose and why?
2. An e-commerce application experiences sudden campaign spikes. Explain how Auto Scaling and load balancing would help.
3. A company wants web servers reachable from the internet but databases hidden from direct exposure. Design the subnet layout and justify it.
4. A media platform stores large video files, user metadata, and cached session data. Which storage and database styles fit each need?

5. A team wants consistent deployment across developer laptops, testing, and production. Why are containers helpful?

6. A workload needs fast key-based lookups at internet scale with flexible schema. Would RDS or DynamoDB be more suitable? Explain.

7. A file uploaded by a user must automatically trigger image processing. Explain how serverless architecture can solve this.

8. A cloud bill is rising unexpectedly. Which Week 11 tools and practices would you use to diagnose and control it?

9. A team wants safer releases with the ability to limit user exposure to new code. Which deployment strategy would you recommend and why?

10. A company wants reproducible cloud environments across development and production. Explain how Terraform helps.

## 17.6 Long Explanatory Questions

1. Explain the major quality attributes of cloud architecture and show how they affect real system design.

2. Discuss AWS foundational security controls including IAM users, groups, roles, policies, MFA, access keys, and organizations.

3. Explain compute and networking design on AWS by connecting EC2, EBS, Auto Scaling, load balancing, VPC, subnets, routing, and gateways.

4. Explain how containers, registries, and Kubernetes together support modern cloud-native application delivery.

5. Discuss cloud storage and database choices using workload-fit reasoning rather than service memorization.

6. Explain serverless architecture on AWS, including Lambda, event sources, statelessness, cost behavior, and monitoring.

7. Discuss cloud observability, auditing, and cost optimization with reference to CloudWatch, CloudTrail, budgets, tagging, and rightsizing.

8. Explain DevOps in the cloud and show how version control, CI/CD, and deployment strategies improve delivery speed and reliability.

9. Discuss infrastructure as code and explain how Terraform translates desired state into managed cloud infrastructure.

## 17.7 Questions Closest to the Sample Paper Style

1. Explain the concept of operational expenditure in cloud computing. How does the shift from CapEx to OpEx affect financial planning and scalability?
2. Compare rehosting, replatforming, and refactoring with respect to effort, cost, and long-term benefit.
3. An application must be deployed in a secure and highly controlled cloud environment. Explain how security groups, network ACLs, private IP addressing, and a bastion-style controlled access path can be used.
4. Explain the difference between horizontal scaling and vertical scaling. Discuss when each is more suitable in cloud environments.
5. Explain the lifecycle of a container from image creation to execution and termination. Also explain the importance of image versioning.
6. Why is service discovery important in distributed and containerized applications?
7. Explain the concept of CI/CD pipelines and describe the usual stages involved. How do they improve software quality and delivery speed?
8. Discuss the importance of infrastructure automation in modern DevOps. Explain how Terraform or similar IaC tools improve consistency and reduce manual errors.

# 18 Final Revision Sheet

## 18.1 Most Important One-Line Definitions

- Cloud computing is on-demand delivery of computing services over the internet with pay-per-use and elasticity.
- EC2 is AWS virtual machine compute.
- EBS is persistent block storage for EC2.
- A VPC is a private logical network in AWS.
- A security group is a stateful virtual firewall at resource level.
- A container packages application code with dependencies.
- Kubernetes orchestrates containerized workloads.
- S3 is object storage.
- RDS is managed relational database service.
- DynamoDB is a managed NoSQL key-value style service.
- Lambda is serverless event-driven compute.

- CloudWatch observes metrics, alarms, and logs.
- CloudTrail records account activity and API actions.
- DevOps is collaborative, automated software delivery and operation.
- IaC means infrastructure is defined through code.

## 18.2 High-Yield Comparison Memory Aids

- Scalability = capacity growth; elasticity = automatic scale up and down.
- High availability = quick recovery; fault tolerance = no visible interruption.
- Security groups = stateful resource firewall; NACLs = stateless subnet filter.
- Containers = lighter and faster; VMs = heavier and more isolated.
- S3 = object, EBS = block, EFS = shared file.
- Relational = structure and transactions; NoSQL = scale and flexible access patterns.
- EC2 = long-running VM control; Lambda = event-driven execution without server management.
- Blue-green = environment switch, rolling = gradual replacement, canary = limited traffic exposure first.
- Declarative = describe desired state; imperative = describe steps.

## 18.3 Mini Formula and Logic Recap

- Address block size intuition: $2^{(32-\text{prefix})}$
- Example: '/24' gives 256 addresses.
- Autoscaling logic: rise in demand $\Rightarrow$ add instances, fall in demand $\Rightarrow$ remove instances.
- Cost logic: idle capacity $\Rightarrow$ waste, rightsizing $\Rightarrow$ better cost-performance balance.
- Policy logic: explicit deny $>$ explicit allow $>$ implicit deny.

## 18.4 Common Confusion Points

- Hybrid cloud is not the same as multi-cloud.
- Elasticity is not just growth; it includes automatic shrink behavior.
- Availability is not the same thing as reliability.

- Security groups and NACLs are not interchangeable.
- Public subnet does not mean every instance in it must be public-facing.
- S3 is not a normal mounted file system.
- DynamoDB is not a general replacement for all relational workloads.
- Serverless does not mean there are no servers.
- Logs, metrics, and CloudTrail records are different observability and governance signals.
- Terraform state is not optional bookkeeping; it is central to safe IaC operation.

## 18.5 Last-Minute Revision by Week

- Week 1: definition, key traits, service models, deployment models, adoption reasons.
- Week 2: scalability, elasticity, HA vs FT, multi-tenancy, well-architected pillars.
- Week 3: AWS vs Azure vs GCP, service equivalents, workload fit, cost/compliance/global reach.
- Week 4: IAM users, groups, roles, policies, MFA, access keys, Organizations.
- Week 5: EC2, AMI, instance families, EBS, snapshots, Auto Scaling, load balancing.
- Week 6: VPC, CIDR, public/private subnets, routing, gateways, NACL vs security groups.
- Week 7: containers, Dockerfile, registry, ECR, Kubernetes basics.
- Week 8: S3, EBS, EFS, storage classes, lifecycle rules, storage security.
- Week 9: relational vs NoSQL, RDS, backups, read replicas, DynamoDB fit.
- Week 10: serverless, Lambda, event sources, statelessness, API Gateway, S3 triggers.
- Week 11: CloudWatch, CloudTrail, budgets, Cost Explorer, rightsizing, tagging.
- Week 12: DevOps, Git, GitHub, CI/CD, pipeline triggers, deployment strategies.
- Week 13: IaC, declarative approach, Terraform workflow, provider, state, apply flow.

## 18.6 Final Exam Advice

- Start with a definition.
- Then explain how it works.
- Add one practical use case.
- Add one trade-off or limitation.
- For comparison questions, prefer a table.
- For architecture questions, explain traffic flow or control flow step by step.
- For scenario questions, state the workload requirement before naming the service.